

Remediation Response

Virtual Card Solution — Vulnerability Assessment Report v1.0 (18/09/2026)

Responding team	Virtual Card Solution — Application Development
Addressed to	Cybersecurity Department, NIB International Bank S.C
Source report	Vulnerability Assessment Report on Virtual Card Solution, v1.0, dated 18/09/2026
Findings addressed	VA-001 to VA-015 (15 findings)
Response date	19/09/2026
Code branch	prepaid-v2
Classification	Confidential

1. Summary

All ten Software Composition Analysis (SCA) findings and three of the four application-layer findings have been remediated in the application source. Dependency advisories across the project have been reduced from 34 to 2, with **no Critical and no High severity advisories remaining**. The two residual advisories affect build-time tooling only and do not ship to the running application.

Two findings are returned for reconsideration on technical grounds (VA-003 and VA-010), one contains a version error that would have left the platform exposed had it been applied literally (VA-011), and one remains open pending a feasibility assessment (VA-015).

IMPORTANT — RETESTING WILL FAIL UNTIL DEPLOYMENT COMPLETES

All remediation described in this response exists in the application source code and has not yet been deployed to `vctestminiapp.nibbank.com.et`. A retest performed against the assessment environment before deployment will reproduce every finding. We will confirm in writing when the environment is ready for validation.

2. Remediation status

Finding	Title	Severity	Status	Outcome
VA-001	Next.js 16.3.2	Critical	REMEDIATED	Upgraded to 16.3.5
VA-002	Undici 7.29.0	Critical	REMEDIATED	7.29.1 — also §7.2
VA-003	Unrestricted file upload	Critical	DISPUTED	Not reproducible — §4.1
VA-004	ReDoS in Tiptap Core	High	REMEDIATED	Upgraded to 3.31.3
VA-005	Nodemailer 9.0.5	High	REMEDIATED	Upgraded to 9.1.1
VA-006	fast-uri 3.1.5	High	REMEDIATED	Upgraded to 3.1.8
VA-007	hono 4.13.3	High	REMEDIATED	Upgraded to 4.13.8
VA-008	mysql2 3.15.3	High	REMEDIATED	Upgraded to 3.24.4
VA-009	Heap overflow in sharp	High	REMEDIATED	0.35.4 — dup. of VA-001
VA-010	xlsx 0.20.3	High	DISPUTED	Already patched — §4.2
VA-011	Prototype pollution in Tiptap	Medium	REMEDIATED	Report error — §4.3
VA-012	Concurrent session management	Medium	REMEDIATED	Single-session enforced
VA-013	Password change leaves sessions active	Medium	REMEDIATED	Forced re-auth — §6
VA-014	Weak password policy	Medium	REMEDIATED	Unified policy + blocklist
VA-015	CSP unsafe-inline in style-src	Low	OPEN	Feasibility — §7.1

Findings marked *Disputed* are returned for reconsideration with technical justification in §4; no remediation was applied because none was found to be required. VA-011 is remediated, but its published recommendation contains a version error that we ask be corrected (§4.3).

3. Dependency remediation (VA-001, 002, 004–009, 011)

Package	Assessed	Required	Installed	Finding
next	16.3.2	≥ 16.3.3	16.3.5	VA-001
undici	7.29.0	≥ 7.29.1	7.29.1	VA-002
@tiptap/core	3.30.2	≥ 3.30.5	3.31.3	VA-004, VA-011
nodemailer	9.0.5	≥ 9.1.1	9.1.1	VA-005
fast-uri	3.1.5	≥ 3.1.7	3.1.8	VA-006
hono	4.13.3	≥ 4.13.5	4.13.8	VA-007
mysql2	3.15.3	≥ 3.23.1	3.24.4	VA-008
sharp	0.35.3	≥ 0.35.4	0.35.4	VA-001, VA-009

Verification across the full dependency tree (473 packages):

```
Before remediation: 34 advisories (1 critical, 5 high, 29 moderate, 1 low)
After remediation:  2 advisories (0 critical, 0 high, 1 moderate, 1 low)
```

The two residual advisories are `baseline-browser-mapping` (moderate) and `postcss-selector-parser` (low). Both are build-time tooling that is not included in the production runtime, and neither has a fixed release available. We propose these be accepted as residual risk.

Note on VA-001 and VA-009 (sharp)

VA-009 correctly cautions that the dependency tree must be verified after upgrading Next.js. This was material: the project pinned `sharp` via an `npm overrides` entry, and upgrading Next.js alone would *not* have moved it. The override floor was raised explicitly. Teams applying VA-001 without VA-009's caveat would have believed the AVIF/libheif decoding path was closed when it was not.

Note on VA-006, VA-007 and VA-008 (Prisma CLI dependency tree)

These three packages reach the project only through the Prisma CLI (`prisma` → `@prisma/dev` → ...) and are not imported by application code. The documented impacts are not reachable in this application: `hono` never serves a request (the server is Next.js), `fast-uri` performs no URI validation for the application, and `mysql2` is a driver for a database engine this application does not use — the Prisma datasource provider is `postgresql`.

All three were nonetheless upgraded. We are separately evaluating moving the Prisma CLI to development dependencies, which would remove this entire sub-tree from production installations.

4. Findings returned for reconsideration

4.1 VA-003 — Unrestricted File Upload via Double Extension Bypass

We are unable to reproduce this finding as described, and request that it be re-evaluated.

- **The test file was a valid spreadsheet.** The filename `customer-mapping-template (copy 1).exe.xlsx` terminates in `.xlsx`; the `.exe` is medial and is not the effective extension. The request body in the evidence capture begins with the ZIP local file header `PK`, followed by `_rels/workbook.xml.rels` and `xl/worksheets/sheet1.xml` — a genuine OOXML workbook. The HTTP/2 200 response parsing it as a spreadsheet is correct behaviour.
- **No uploaded file is ever written to disk.** The application persists no upload to any filesystem location. The buffer is parsed in memory and discarded. Consequently the recommendations to “store uploads outside web roots” and “disable script execution in upload directories” have no applicable target — there is no upload directory.
- **The execution mechanism does not exist on this platform.** Double-extension exploitation depends on a web server resolving a non-final extension (for example Apache `mod_mime` `multiview`). Next.js does not dispatch on file extension.
- **The referenced CVEs are for unrelated technology stacks.** CVE-2017-9841 (PHPUnit), CVE-2021-41773 (Apache HTTP Server) and CVE-2023-50164 (Apache Struts) are not reachable from a Next.js/Node.js application.
- **The CVSS vector overstates privilege requirements.** The vector specifies `PR:N`. Both upload endpoints require an authenticated `SUPER_ADMIN` session and return HTTP 401 otherwise; the evidence capture itself shows an authenticated administrative referer. `PR:H` would be accurate.

We note the Prioritized Remediation Table on page 31 records VA-003 as *Solved* (16/09/2026) while the detailed finding on page 14 records it as *Open*. We would appreciate clarification of the intended status.

RELATED DEFECT IDENTIFIED AND REMEDIATED

While investigating VA-003 we identified a genuine server-side input validation weakness in the same feature, which was not reported by the assessment. The customer-mapping migration endpoint accepted the client's own validation verdict: a row submitted with `status: "valid"` was written to the database without the server re-deriving that verdict, allowing arbitrary customer-ID mappings to bypass file parsing entirely.

The endpoint now recomputes every row's status server-side from the submitted values, using the same classification routine the file parser uses. Magic-byte validation and a decompression bound were added to both spreadsheet upload endpoints at the same time. Details available on request.

4.2 VA-010 — Multiple Security Vulnerabilities in `xlsx` 0.20.3

Both cited vulnerabilities were already fixed in the version under assessment:

CVE	Affects	Fixed in	Installed
CVE-2023-30533	through 0.19.2	0.19.3	0.20.3
CVE-2024-22363	through 0.20.1	0.20.2	0.20.3

The likely cause of the false positive is that the npm registry package `xlsx` is frozen at 0.18.5; SheetJS moved distribution to its own CDN from 0.18.6 onward. Scanners matching on package name against npm advisory data will flag any installed version, because no fixed release exists *on npm*.

This project does not install from npm. `package.json` declares the vendor CDN tarball directly, and `package-lock.json` pins it by integrity hash:

```
"xlsx": "https://cdn.sheetjs.com/xlsx-0.20.3/xlsx-0.20.3.tgz"
integrity: sha512-oLDq3jw7AcLqKWH2AhCpVTZ18mf6X2YReP+Neh0SJUzV/BdZYjth94tG5toiMB1PPrYtxOCfaoUCkvtuH+3AJA==
```

VA-010's recommendation — “obtaining a security-fixed distribution from an authoritative source after validating its provenance” — describes the control that was already in place before the assessment.

We accept the report's accompanying observation that this application *does* parse untrusted spreadsheets, so the mitigating condition it describes does not apply. The hardening noted in §4.1 was implemented for that reason.

4.3 VA-011 — Version error in recommendation

ACTION REQUESTED — REPORT CORRECTION

VA-011 recommends upgrading `@tiptap/core` to **3.30.4 or higher**. Version 3.30.4 is itself vulnerable. VA-004, describing the same advisory pair in the same package, correctly states the affected range as `>=3.7.0 <3.30.5` and recommends **3.30.5**.

An implementer following VA-011 literally would pin 3.30.4, close the finding, and remain exposed to both the prototype pollution and the ReDoS. We recommend VA-011 be corrected before the report is distributed further.

This project is unaffected: the tree was taken to 3.31.3. We would also note that VA-011 rates the prototype pollution as Medium (5.3) while the corresponding advisory database entry rates the package High. In this application the affected component renders administrator-authored content to every card requester, so we treated it with the higher rating.

4.4 Duplicate findings

For tracking purposes, three findings describe defects already covered elsewhere in the report and are closed by a single remediation each: VA-009 duplicates the `sharp/libheif` component of VA-001, and VA-011 duplicates VA-004.

5. Application-layer remediation

5.1 VA-012 — Concurrent session management

A `tokenVersion` counter was added to the user record. Each issued session token carries the value it was signed against, and token verification rejects any token whose value no longer matches. Incrementing the counter therefore terminates every active session for that account in one operation.

Recommendation	Status
Limit users to a single active session	IMPLEMENTED
Invalidate older sessions when a new session is established	IMPLEMENTED
Provide a log out from all devices feature	MECHANISM READY; UI CONTROL NOT YET ADDED
Notify users when a new login occurs	NOT IMPLEMENTED
Display active sessions to users	NOT IMPLEMENTED

Signing in now terminates any session the account holds on another browser or device. This applies to staff roles (Super Admin, Maker, Checker); Guest MiniApp sessions use a separate mechanism and are unaffected.

Session lifetime correction

During this work we identified that the session timeout control was not being enforced as documented. The session cookie was set to expire after 15 minutes, annotated as satisfying PCI DSS 8.2.8, but the underlying token was independently configured with a one-hour validity. Because the server enforces the token's own expiry, the effective session lifetime was **four times the advertised value**, with the 15-minute limit enforced only by the browser.

Token validity is now 15 minutes, aligned with the cookie, and the cookie lifetime is derived from the token itself so the two values cannot diverge again. Session renewal on activity was also corrected so that the 15 minutes operates as an inactivity timeout, as PCI DSS 8.2.8 specifies, rather than an absolute cap that would interrupt legitimate administrative work.

5.2 VA-013 — Password change does not invalidate active sessions

Confirmed and remediated. Changing or resetting a password now increments `tokenVersion` within the same database transaction as the credential update, terminating all sessions for the account. The session performing the change is also terminated and the user is returned to the login screen, satisfying the recommendation to force re-authentication after credential updates.

Applying the same reasoning, we extended this to email address changes. The email address is the login identifier for this application, so changing it constitutes a credential change and now terminates all sessions in the same way.

Recommendation	Status
Invalidate all active sessions when a user changes their password	IMPLEMENTED
Force re-authentication after credential updates	IMPLEMENTED
Rotate session identifiers upon password change	IMPLEMENTED

Recommendation	Status
Maintain server-side session revocation tracking	IMPLEMENTED
Notify users of all active sessions / allow manual termination (<i>optional</i>)	NOT IMPLEMENTED

5.3 VA-014 — Weak password policy

Confirmed and remediated. Investigation established that the root cause was a policy divergence rather than a uniformly weak policy. The application enforced two different policies:

Rule	Registration / change	Password reset
Minimum length	10	8
Uppercase required	Yes	No
Lowercase required	Yes	No
Special character required	Yes	No
Weak-pattern blocklist	Yes	No

The password reset flow — reachable after an email compromise — enforced the weaker of the two. Both example passwords cited in the finding (Test@123, Admin@123) are below the 10-character minimum and would have been rejected by the primary policy; they were accepted because the reset path did not apply it.

All three password entry points now share a single policy implementation. The blocklist was extended to reject:

- **Common base words** after stripping case, digits and punctuation — this is what catches the Word@123 construction, which satisfies every composition rule while remaining trivially guessable.
- **Character-substitution variants**, so P@ssw0rd is rejected on the same term as password.
- **Institution-specific terms** including nib, nibbank, vcard and ethiopia, addressing the CompanyName@123 pattern named in the recommendation.
- **Seasons and month names**, and **predictable sequences** — sequential, repeated or same-keyboard-row runs of four or more characters.

The policy was tested against fifteen weak passwords, including both examples cited in the finding, and against five representative strong passwords to confirm no false rejections.

Two recommendations were not implemented and we would welcome the Department's guidance:

- **Breach-database (HIBP-style) checking.** Technically straightforward using the k-anonymity API, but it introduces an outbound call to a third-party service on the authentication path. We consider this a network egress and compliance decision for the Department rather than an application decision.
- **Passphrase encouragement via relaxed composition rules.** NIST SP 800-63B, cited in the finding, recommends length and blocklisting *in place of* composition rules. Implementing this properly means exempting long passphrases from the symbol and case requirements. We did not relax an existing control unilaterally, and would need confirmation that this aligns with the Bank's password standard.

6. Retesting guidance

VA-013 — PLEASE NOTE THE TEST METHOD

The reproduction steps for VA-013 do not specify which browser performs the password change. If the change is performed in Browser A and Browser A is then checked, the tester will observe an active session and may record the finding as unresolved.

This is expected: the user performing the change is returned to the login screen and signs in again, which is the forced re-authentication the finding requests. To validate that *other* sessions are terminated, please sign in to the same account in two browsers, change the password in Browser B, and confirm that Browser A is rejected and redirected to login.

For VA-012, single-session enforcement can be validated by signing in to the same account in two browsers sequentially and confirming the first is terminated when the second authenticates.

For VA-014, we suggest testing the reset flow specifically in addition to the change-password form, as that path carried the defect.

7. Open items

7.1 VA-015 — CSP `unsafe-inline` in `style-src`

Accepted as a valid hardening observation and currently open. The directive is present because the application's UI component libraries inject inline styles at runtime; removing it is not a configuration change alone. We are assessing whether a nonce or hash-based policy is achievable without regression, and will report back with either a remediation plan or a documented risk acceptance request.

We note the finding's own characterisation that this is a defence-in-depth concern rather than a directly exploitable weakness, and that the policy is otherwise strict — script execution is nonce-gated, `unsafe-eval` is excluded in production, and `frame-ancestors` is set to `'none'`.

7.2 Infrastructure actions outside the application codebase

The following require action by infrastructure and deployment owners and cannot be completed by an application code change. We raise them here because two materially affect findings in this report.

Node.js runtime — upgrade to 24.21.0 or later

This materially affects VA-002. Node.js embeds its own internal copy of `undici`, which serves every outbound `fetch` call the application makes — including payment initiation, payment status, account lookup and prepaid customer services. The runtime in use bundles `undici` 7.18.2, which is *older* than the 7.29.0 version the assessment flagged.

Upgrading the npm `undici` package, as VA-002 recommends, does not affect this internal copy. Node.js 24.21.0 (released 08/09/2026) is the first release bundling `undici` 7.29.1. Without this runtime upgrade, VA-002 remains only partially remediated in the deployed environment.

Session token lifetime — set `JWT_EXPIRES_IN=15m`

Required for the session timeout correction described in §5.1 to take effect. The application reads this value from the deployment environment; until it is set, the deployed service will continue to issue tokens with the previous validity period.

SMTP transport security — confirm `MAIL_ALLOW_SELF_SIGNED` is not enabled

This was not reported by the assessment, and we raise it because we consider it material. The application exposes a configuration flag that, when enabled, disables TLS certificate validation on the SMTP connection. With validation disabled, password reset links, email change verification tokens and Guest one-time codes traverse a connection on which no certificate is verified, presenting an interception risk on a path that carries account recovery material.

We request confirmation of the value configured in the deployed environment. If it is not required operationally, we propose removing the flag entirely so it cannot be enabled inadvertently.

8. Requested actions

1. Reconsider **VA-003** and **VA-010** on the technical grounds set out in §4.
2. Correct the version in **VA-011** (3.30.4 → 3.30.5) before further distribution.
3. Clarify the conflicting status recorded for **VA-003** (page 14 “Open” / page 31 “Solved”).
4. Confirm the retest window, noting that remediation must be deployed to the assessment environment first.
5. Advise on breach-database checking and passphrase policy (§5.3).
6. Confirm the deployed value of `MAIL_ALLOW_SELF_SIGNED` (§7.2).

Prepared by — Application Development

Reviewed by — EPMO

This response addresses Vulnerability Assessment Report v1.0 dated 18/09/2026 for the Virtual Card Solution. Remediation described herein is present in application source on branch `prepaid-v2` and is pending deployment to the assessment environment. Dependency figures were produced by `npm audit` across 473 resolved packages. Classification: Confidential — handle and store in accordance with the source report's distribution controls.